Linux Process Watchdog (Nexus) – Project Documentation

Oct 17, 2026 · @ak

1. Overview

Nexus is a Linux process watchdog: it polls registered processes, detects when one dies, restarts it under a bounded policy, verifies the restart, and records the whole incident. It pairs a FastAPI backend with a React operations dashboard.

Problem it solves. A long-running service can stop without warning (out-of-memory kill, segmentation fault, unhandled exception, resource exhaustion, an accidental `kill`). Without a watchdog it stays down until an administrator notices, restarts it by hand and works out what happened.

Team. Bhanu Jakka (2520030006), Akshita Anand (2520030043), A. Kuvira (2520030192), P. Sudha (2520030455).

Key features

- Background monitor loop that checks every registered process (default every 5 s, minimum 2 s).
- Identity-checked detection: PID, process start time (within 2 s) and command line must all match; zombies count as not alive.
- Automatic restart limited by a per-application attempt cap (default 3) and a cooldown between attempts.
- Recovery counts as successful only if the process is running within 3 s and the optional HTTP health check returns 200–399.
- Incidents with a timeline, restart-attempt records and alerts, all stored in SQLite.
- Dashboard with two explicit modes: **LIVE MODE** (backend reachable) and **DEMO MODE** (simulated, browser-only, never labelled as live).
- Self-protection: only the allowlisted `backend/test_process.py` can be started or restarted, and the frontend can never send a shell command.

2. Architecture

The React dashboard talks to the FastAPI backend over REST (`/api`); inside the backend, an async monitor loop drives a process checker, a restart manager and a policy engine, and everything is persisted in SQLite.

Component	Location	Responsibility
React dashboard	`src/` (Vite, React, TypeScript, Recharts)	Overview, Process Monitor, Applications, Incidents, Recovery Policies, Analytics, Alerts, Settings. Switches between LIVE and DEMO mode.
API layer	`backend/app/main.py`	REST endpoints, CORS, lifespan start of the monitor loop.
Monitor loop	`monitor_loop()` in `main.py`	Async task that calls `poll_once()` every interval (default 5 s).
Process checker	`RecoveryService.is_alive()` in `recovery.py`	psutil check of PID, start time, command line and zombie state.
Restart manager	`RecoveryService.spawn()` / `attempt_recovery()`	Starts the allowlisted script, waits up to 3 s, runs the health check.
Policy engine	`attempt_recovery()`	Enforces enabled flag, attempt limit and cooldown.
Persistence	`backend/app/db.py` (SQLAlchemy)	SQLite tables listed below.
Test target	`backend/test_process.py`	Disposable worker that sleeps in a loop and exits cleanly on SIGTERM/SIGINT.

Data model

Table	Holds
`applications`	Registered process: name, PID, start time, status, health-check URL, restart settings, `expected_running`, `restart_count`.
`recovery_policies`	One per application: enabled, maximum attempts, cooldown seconds, health check, backoff label.
`incidents`	Type, severity, status (Open, Recovering, Recovered, Unresolved), detection and recovery times, duration, attempts, JSON timeline.
`restart_attempts`	Attempt number, start/finish time, outcome (Started, Succeeded, Failed), error details.
`alerts`	Title, message, severity, acknowledged/read flags, linked incident.
`monitoring_history`	Status, CPU % and memory (MB) sampled each poll; feeds analytics.

Monitoring and recovery flow

1. The backend starts and launches the monitor loop.
2. Each cycle, every application with monitoring enabled is checked.
3. A process that is missing, a zombie, or has the wrong identity is marked **Failed** (unless `expected_running` is false, in which case it is **Stopped** and no incident is raised).
4. A new incident is opened with a "Failure detected" timeline event.
5. The policy engine checks the attempt limit and cooldown, then the allowlisted script is restarted.
6. Recovery is verified (alive within 3 s plus health check), the incident is closed as **Recovered**, and an alert is raised. If verification fails and attempts are exhausted, the incident becomes **Unresolved** with a Critical alert.

Safety design

- Only `test_process.py` can be started; registration with any other script is rejected with HTTP 422.

- Frontend requests never carry shell commands; process arguments must be short strings.
- PID reuse is handled by matching start time and command line, not PID alone.

3. Installation and setup

Nexus runs on Ubuntu or Debian with Python 3 and Node.js; no secrets are needed for the demo.

Prerequisites

- Linux (Ubuntu / Debian), Python 3 with `venv`, Node.js and npm.
- Ports 8000 (API) and 3000 (dashboard) free.

Backend

```
cd backend
python3 -m venv .venv
. .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Interactive API docs open at `http://127.0.0.1:8000/docs`. The database file `backend/nexus.db` is created automatically on first start.

Frontend

```
npm install
npm run dev
```

The dashboard listens on port 3000 and Vite proxies `/api` to port 8000. It switches to LIVE MODE when the API answers and falls back to DEMO MODE when it does not. To point at another API: `VITE_API_URL=http://127.0.0.1:8000/api npm run dev`.

Environment variables

Variable	Default	Purpose
`NEXUS_DATABASE_URL`	`sqlite:///./nexus.db`	Database connection string.
`NEXUS_MONITOR_INTERVAL_SECONDS`	`5`	Seconds between monitor cycles (values below 2 are raised to 2).
`NEXUS_CORS_ORIGINS`	`http://localhost:3000,http://127.0.0.1:3000`	Allowed browser origins.
`VITE_API_URL`	`/api`	API base URL used by the dashboard.

Build, typecheck and tests

```
npm run typecheck
npm run build
cd backend && pytest
```

4. Usage and configuration

Run a live recovery demonstration

1. Start the backend and the dashboard.
2. Register the safe test worker and start it:

```
curl -X POST http://127.0.0.1:8000/api/demo/controlled-test \
  -H 'Content-Type: application/json' \
  -d '{"name":"nexus-test-worker","auto_start":true}'
```

3. Stop it to simulate a crash: click **Run live recovery** on the Overview page, or call `POST /api/demo/live-recovery`.

4. On its next poll the monitor detects the failure, opens an incident, restarts the worker, verifies it and raises a "Recovery succeeded" alert.
5. Inspect the result under Incidents and Alerts.

The **Simulated demo** button runs the same story in the browser only (Application Healthy, Failure Detected, Recovery Attempt, Recovery Verified, Application Healthy). Settings has a reset for the sample data.

Dashboard pages

Page	Purpose
Overview	Totals, health distribution, incident timeline, recent incidents, recovery activity, system status.
Process Monitor	Searchable, filterable table: name, PID, status, CPU, memory, uptime, health check, restart count.
Applications	Register, edit, inspect, enable or disable monitoring.
Incidents	Severity, status, attempts, recovery duration, event timeline.
Recovery Policies	Automatic restart, maximum attempts, cooldown, health check.
Analytics	Recovery duration, success rate, failure counts; shows "Insufficient data" until enough history exists.
Alerts	Filter, acknowledge, jump to the linked incident.
Settings	Refresh interval, environment, backend status.

Recovery policy settings

Setting	Range	Meaning
`maximum_attempts`	1 to 10 (default 3)	Restart attempts allowed per incident.

Setting	Range	Meaning
`cooldown_seconds`	0 to 86400	Minimum wait between attempts (the demo worker uses 5 s).
`health_check` / `health_check_url`	optional	If set, the URL must answer HTTP 200–399 within 2 s for recovery to count.
`expected_running`	true / false	Set to false for planned stops so no incident is raised.
`monitoring_enabled`	true / false	Excludes the application from polling.

REST API reference

Method and path	Purpose
`GET /api/health`	Backend and monitoring-agent status.
`GET /api/dashboard/summary`	Application counts, incidents in 24 h, MTTR.
`GET /api/applications`, `POST /api/applications`	List or register an application (only `test_process.py` accepted).
`GET`, `PUT`, `DELETE /api/applications/{id}`	Read, update or remove one application.
`GET /api/applications/{id}/metrics` and `/incidents`	Metrics and incidents for one application.
`POST /api/applications/{id}/restart`	Controlled manual restart that creates an incident.
`POST /api/demo/controlled-test`	Create (and start) the allowlisted test worker.
`POST /api/demo/live-recovery`	Stop the worker so the monitor recovers it.
`GET /api/incidents`, `GET /api/incidents/{id}`	List (filter by `status`, `severity`) or read incidents.
`GET`, `POST`, `PUT /api/recovery-policies`	List, create or update recovery policies.

Method and path	Purpose
`GET /api/alerts`, `POST /api/alerts/{id}/acknowledge`	List alerts or acknowledge one.
`GET /api/analytics/summary`	MTTR, success rate, failure frequency, detection latency, restart attempts, unresolved count.

5. Code reference, testing and troubleshooting

Source layout

Path	Contents
`backend/app/main.py`	FastAPI app, REST routes, monitor loop, lifespan startup.
`backend/app/recovery.py`	`RecoveryService`: identity check, metrics, spawn, stop, health check, recovery attempts, `poll_once()`.
`backend/app/db.py`	SQLAlchemy models and engine setup.
`backend/app/schemas.py`	Pydantic request models and the script allowlist validator.
`backend/app/config.py`	Environment-based settings.
`backend/test_process.py`	Disposable worker used as the recovery target.
`backend/tests/test_api.py`	pytest suite (3 tests).
`src/App.tsx`, `src/api.ts`, `src/data.ts`, `src/types.ts`	Dashboard, API client, demo data and types.

Testing

The pytest suite covers: health endpoint with the agent running; registration of the controlled worker; rejection of an arbitrary script (HTTP 422); and a manual controlled restart that ends in an incident with status Recovered. All 3 tests passed in our local run.

Abrupt-kill detection was exercised with curl against a live server. Crash loops and simultaneous failures are not yet tested.

Troubleshooting

Symptom	Likely cause and fix
Dashboard shows DEMO MODE	Backend not reachable. Start uvicorn on port 8000 and check `NEXUS_CORS_ORIGINS` or `VITE_API_URL`.
`422` when registering an application	Only `test_process.py` is accepted as the script.
`409` on registration	An application with that name already exists.
`403` on manual restart	The application is not the allowlisted controlled worker.
`409` from live-recovery	The worker is not running; start it with `/api/demo/controlled-test`.
Incident stays Unresolved	Attempt limit reached or health check URL unreachable; fix the URL or raise `maximum_attempts`.
Analytics show "Insufficient data"	Availability needs a full observation window of monitoring history.

Known limitations

- Only the allowlisted `test_process.py` worker can be restarted; real services are future work.
- Automatic retry up to the policy limit is not yet implemented: after a failed first attempt the incident stays open and is not retried by the monitor.
- Application and policy edit forms in the dashboard are not yet synchronised with the backend write endpoints.
- No authentication or roles; single host only.
- The dashboard summary endpoint needs a fix (listed as a planned item).